

AI Assisted Coding

Name : M.PRAVALIKA

H.no : 2303A52347

Batch : 34

Course Code : 23CS002PC304

Assignment Number : 8.2

Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases

Aim

To apply Test-Driven Development (TDD) using AI by generating test cases first and then implementing Python code that satisfies those tests.

Task Description

Task 1 – Test-Driven Development for Even/Odd Number Validator

Prompt

Generate unittest test cases for a Python function `is_even(n)` that checks whether a number is even. Handle integers, zero, negative values, and invalid inputs.

The screenshot shows a Jupyter Notebook titled 'Untitled7.ipynb'. The first code cell contains the definition of the `is_even` function:

```
[1] def is_even(n):
    """Checks if a number is even."""
    if not isinstance(n, int):
        raise TypeError("Input must be an integer")
    return n % 2 == 0
```

The second code cell contains the `unittest` test cases for the `is_even` function:

```
[2] import unittest

class TestIsEvenFunction(unittest.TestCase):

    def test_positive_even(self):
        self.assertTrue(is_even(4))
        self.assertTrue(is_even(100))

    def test_positive_odd(self):
        self.assertFalse(is_even(3))
        self.assertFalse(is_even(99))

    def test_zero(self):
        self.assertTrue(is_even(0))

    def test_negative_even(self):
        self.assertTrue(is_even(-2))
        self.assertTrue(is_even(-10))

    def test_negative_odd(self):
        self.assertFalse(is_even(-5))
        self.assertFalse(is_even(-13))

    def test_invalid_input_string(self):
        with self.assertRaises(TypeError):
            is_even("hello")
```

Below the code, there is a text prompt: "Now, let's create the `unittest` test cases for the `is_even` function."

The Gemini chat interface on the right shows the following interaction:

Task 1 - Even/Odd Number Validator Prompt Generate `unittest` test cases for a Python function `is_even(n)` that checks whether a number is even. Handle integers, zero, negative values, and invalid inputs.

Okay, I will first provide the `is_even` function, and then generate `unittest` test cases for it, covering positive, negative, zero, and invalid inputs.

Great news! All 9 of your `is_even` function tests passed successfully, as indicated by the 'OK' in the output. This means your function behaves as expected for all the scenarios you've tested. You might notice a `DeprecationWarning` about `unittest.makeSuite()`; this means that function is older and might be removed in a future Python version. For now, it works fine, but in newer code, you might want to consider using `unittest.TestLoader.loadTestsFromTestCase()` as suggested in the warning.

The task of generating `unittest` test cases for the `is_even` function has been completed, and all tests passed successfully. If you have any further functions you'd like to test or other tasks, please let me know!

The task of generating and running `unittest` test cases for the `is_even` function has been successfully completed. All tests passed, confirming the

Buttons: Refactor test runner to use 'unittest.TestLoader', Explain the 'DeprecationWarning' in the output, Add more test cases for edge scenarios for 'is_even'.

What can I help you build? Gemini 2.5 Flash

The screenshot shows the Jupyter Notebook with the code from the previous image. The third code cell contains the following code:

```
[3] self.assertFalse(is_even(-13))

def test_invalid_input_string(self):
    with self.assertRaises(TypeError):
        is_even("hello")

def test_invalid_input_float(self):
    with self.assertRaises(TypeError):
        is_even(3.14)

def test_invalid_input_none(self):
    with self.assertRaises(TypeError):
        is_even(None)

def test_invalid_input_list(self):
    with self.assertRaises(TypeError):
        is_even([1, 2])

# To run the tests, uncomment the following lines:
# if __name__ == '__main__':
#     unittest.main(argv=['first-arg-is-ignored'], exit=False)
```

Below the code, there is a text prompt: "To run these tests, you can uncomment the `unittest.main()` call in the last code cell and execute it. Alternatively, you can run them directly in a Colab environment using `unittest.main()` from a separate cell, or by explicitly creating a test suite and running it."

The fourth code cell contains the following code:

```
[4] # Example of how to run the tests in Colab:
# This is an alternative to uncommenting unittest.main() above.
# You can also run individual test methods from the class.

suite = unittest.TestSuite()
suite.addTest(unittest.makeSuite(TestIsEvenFunction))
runner = unittest.TextTestRunner()
runner.run(suite)
```

The output of the fourth code cell shows the test results:

```
/tmp/ipython-input-3894288446.py:6: DeprecationWarning: unittest.makeSuite() is deprecated and will be removed in Python 3.13. Please use
.....
Ran 9 tests in 0.009s

OK
unittest.runner.TextTestResult run=0 errors=0 failures=0
```

The Gemini chat interface on the right shows the following interaction:

Task 1 - Even/Odd Number Validator Prompt Generate `unittest` test cases for a Python function `is_even(n)` that checks whether a number is even. Handle integers, zero, negative values, and invalid inputs.

Okay, I will first provide the `is_even` function, and then generate `unittest` test cases for it, covering positive, negative, zero, and invalid inputs.

Great news! All 9 of your `is_even` function tests passed successfully, as indicated by the 'OK' in the output. This means your function behaves as expected for all the scenarios you've tested. You might notice a `DeprecationWarning` about `unittest.makeSuite()`; this means that function is older and might be removed in a future Python version. For now, it works fine, but in newer code, you might want to consider using `unittest.TestLoader.loadTestsFromTestCase()` as suggested in the warning.

The task of generating `unittest` test cases for the `is_even` function has been completed, and all tests passed successfully. If you have any further functions you'd like to test or other tasks, please let me know!

The task of generating and running `unittest` test cases for the `is_even` function has been successfully completed. All tests passed, confirming the function's correctness across various scenarios. I have nothing further to do on this task.

Buttons: Refactor test runner to use 'unittest.TestLoader', Explain the 'DeprecationWarning' in the output, Add more test cases for edge scenarios for 'is_even'.

What can I help you build? Gemini 2.5 Flash

Explanation

- Validates input type
- Uses modulo operator
- Handles zero, negative, and large integers safely

Task 2 – String Case Converter

Prompt

Generate test cases for `to_uppercase(text)` and `to_lowercase(text)` handling empty strings, mixed case, and invalid inputs.

The screenshot shows a Jupyter Notebook with two cells. The first cell contains the implementation of `to_uppercase` and `to_lowercase` functions. The second cell contains unit tests for these functions using `unittest`.

```
def to_uppercase(text):  
    """Converts a string to uppercase."""  
    if not isinstance(text, str):  
        raise TypeError("Input must be a string")  
    return text.upper()  
  
def to_lowercase(text):  
    """Converts a string to lowercase."""  
    if not isinstance(text, str):  
        raise TypeError("Input must be a string")  
    return text.lower()  
  
Now, let's create the 'unittest' test cases for the 'to_uppercase' and 'to_lowercase' functions.  
  
import unittest  
  
class TestStringCaseConverter(unittest.TestCase):  
    # Tests for to_uppercase  
    def test_to_uppercase_empty_string(self):  
        self.assertEqual(to_uppercase(""), " ")  
  
    def test_to_uppercase_mixed_case(self):  
        self.assertEqual(to_uppercase("HelloWorld"), "HELLOWORLD")  
        self.assertEqual(to_uppercase("python"), "PYTHON")  
  
    def test_to_uppercase_already_uppercase(self):  
        self.assertEqual(to_uppercase("UPPERCASE"), "UPPERCASE")  
  
    def test_to_uppercase_already_lowercase(self):  
        self.assertEqual(to_uppercase("lowercase"), "LOWERCASE")  
  
    def test_to_uppercase_with_numbers_and_symbols(self):  
        self.assertEqual(to_uppercase("Test123!@#"), "TEST123!@#")  
  
    def test_to_uppercase_invalid_input_int(self):  
        with self.assertRaises(TypeError):  
            to_uppercase(123)  
  
    def test_to_uppercase_invalid_input_none(self):  
        with self.assertRaises(TypeError):  
            to_uppercase(None)  
  
    def test_to_uppercase_invalid_input_list(self):  
        with self.assertRaises(TypeError):  
            to_uppercase(["a", "b"])
```

The right sidebar shows the Gemini chat interface with the following text:

Task 2 – String Case Converter AI Prompt Used Generate test cases for `to_uppercase(text)` and `to_lowercase(text)` handling empty strings, mixed case, and invalid inputs.

Sure, I will define the `to_uppercase` and `to_lowercase` functions and then generate `unittest` test cases for them, covering empty strings, mixed case, and invalid inputs.

Great news! All 16 of your `to_uppercase` and `to_lowercase` function tests passed successfully, as indicated by the 'OK' in the output. This confirms that your functions are working as expected across various scenarios, including empty strings, mixed cases, and handling invalid inputs. As with the previous task, you'll see a `DeprecationWarning` for `unittest.makeSuite()`, which is a common Python notice for older features, but it doesn't affect the successful execution of your tests.

Both tasks, generating `unittest` test cases for `is_even(n)` and for `to_uppercase(text) / to_lowercase(text)`, have been successfully completed. All tests passed as expected. If you have any new tasks or questions, feel free to ask!

Both tasks have been successfully completed and verified. I have nothing further to do at this moment. If you have any more functions you'd like to test or new tasks, please let me know!

Refactor tests to use `unittest.TestLoader.loadTestsFromTestCase()`
Generate more unittest cases for a new function
Explain the purpose of the 'unittest' module

What can I help you build?

Gemini 2.5 Flash

The screenshot shows a Jupyter Notebook with two cells. The first cell contains the implementation of `to_uppercase` and `to_lowercase` functions. The second cell contains unit tests for these functions using `unittest`.

```
def to_uppercase(text):  
    """Converts a string to uppercase."""  
    if not isinstance(text, str):  
        raise TypeError("Input must be a string")  
    return text.upper()  
  
def to_lowercase(text):  
    """Converts a string to lowercase."""  
    if not isinstance(text, str):  
        raise TypeError("Input must be a string")  
    return text.lower()  
  
Now, let's create the 'unittest' test cases for the 'to_uppercase' and 'to_lowercase' functions.  
  
import unittest  
  
class TestStringCaseConverter(unittest.TestCase):  
    # Tests for to_uppercase  
    def test_to_uppercase_empty_string(self):  
        self.assertEqual(to_uppercase(""), " ")  
  
    def test_to_uppercase_mixed_case(self):  
        self.assertEqual(to_uppercase("HelloWorld"), "HELLOWORLD")  
        self.assertEqual(to_uppercase("python"), "PYTHON")  
  
    def test_to_uppercase_already_uppercase(self):  
        self.assertEqual(to_uppercase("UPPERCASE"), "UPPERCASE")  
  
    def test_to_uppercase_already_lowercase(self):  
        self.assertEqual(to_uppercase("lowercase"), "LOWERCASE")  
  
    def test_to_uppercase_with_numbers_and_symbols(self):  
        self.assertEqual(to_uppercase("Test123!@#"), "TEST123!@#")  
  
    def test_to_uppercase_invalid_input_int(self):  
        with self.assertRaises(TypeError):  
            to_uppercase(123)  
  
    def test_to_uppercase_invalid_input_none(self):  
        with self.assertRaises(TypeError):  
            to_uppercase(None)  
  
    def test_to_uppercase_invalid_input_list(self):  
        with self.assertRaises(TypeError):  
            to_uppercase(["a", "b"])
```

The right sidebar shows the Gemini chat interface with the following text:

Task 2 – String Case Converter AI Prompt Used Generate test cases for `to_uppercase(text)` and `to_lowercase(text)` handling empty strings, mixed case, and invalid inputs.

Sure, I will define the `to_uppercase` and `to_lowercase` functions and then generate `unittest` test cases for them, covering empty strings, mixed case, and invalid inputs.

Great news! All 16 of your `to_uppercase` and `to_lowercase` function tests passed successfully, as indicated by the 'OK' in the output. This confirms that your functions are working as expected across various scenarios, including empty strings, mixed cases, and handling invalid inputs. As with the previous task, you'll see a `DeprecationWarning` for `unittest.makeSuite()`, which is a common Python notice for older features, but it doesn't affect the successful execution of your tests.

Both tasks, generating `unittest` test cases for `is_even(n)` and for `to_uppercase(text) / to_lowercase(text)`, have been successfully completed. All tests passed as expected. If you have any new tasks or questions, feel free to ask!

Both tasks have been successfully completed and verified. I have nothing further to do at this moment. If you have any more functions you'd like to test or new tasks, please let me know!

Refactor tests to use `unittest.TestLoader.loadTestsFromTestCase()`
Generate more unittest cases for a new function
Explain the purpose of the 'unittest' module

What can I help you build?

Gemini 2.5 Flash

Explanation

- Safe handling for None
- Uses built-in string methods
- Prevents runtime errors

Task 3 – List Sum Calculator

Prompt

Generate test cases for `sum_list(numbers)` that handles empty lists, negative values, and ignores non-numeric items.

The screenshot displays a Jupyter Notebook environment with a code editor on the left and a Gemini AI chat window on the right.

Code Editor:

```
Now, let's create the unittest test cases for the sum_list function.
```

```
import unittest

class TestSumListFunction(unittest.TestCase):

    def test_empty_list(self):
        self.assertEqual(sum_list([]), 0)

    def test_list_with_positive_numbers(self):
        self.assertEqual(sum_list([1, 2, 3, 4, 5]), 15)
        self.assertEqual(sum_list([10, 20, 30]), 60)

    def test_list_with_negative_numbers(self):
        self.assertEqual(sum_list([-1, -2, -3]), -6)
        self.assertEqual(sum_list([-100]), -100)

    def test_list_with_mixed_numbers(self):
        self.assertEqual(sum_list([-1, 0, 1, 2, -5]), -3)
        self.assertEqual(sum_list([5, -2, 0, -4]), 7)

    def test_list_with_floats(self):
        self.assertEqual(sum_list([1.5, 2.5, 3.0]), 7.0)
        self.assertEqual(sum_list([-1.5, 2.0]), 0.5)

    def test_list_with_non_numeric_items_only(self):
        # with the fix, True/False should be ignored
        self.assertEqual(sum_list(['a', 'b', 'c']), 0)
        self.assertEqual(sum_list([None, True, False]), 0)

    def test_list_with_mixed_numeric_and_non_numeric_items(self):
        self.assertEqual(sum_list([1, 'a', 2, None, 3.5]), 6.5)
        self.assertEqual(sum_list(['hello', -5, 10, 'world', 0]), 5)
        self.assertEqual(sum_list([10, {}, [], 20, 'test', -5]), 25)

    def test_list_with_zero(self):
        self.assertEqual(sum_list([0, 0, 0]), 0)
        self.assertEqual(sum_list([0, 5, -5]), 0)
```

To run these tests, you can use `unittest.main()` or create a test suite as shown below.

```
# Example of how to run the tests in Colab:
suite = unittest.TestSuite()
suite.addTest(unittest.makeSuite(TestSumListFunction))
```

Gemini Chat Window:

I have updated the `sum_list` function in cell 5191a239 to correctly ignore boolean values, which were being inadvertently summed as integers. To confirm that this fix has resolved the test failure, please re-execute the `sum_list` function definition cell (5191a239) and then re-run the test suite in cell 2c88878.

I have successfully identified and provided a fix for the `sum_list` function to correctly ignore boolean values when summing. The updated code in cell 5191a239 now explicitly checks for and excludes `bool` types. To verify that this fix resolves the test failure, please re-execute the updated `sum_list` function definition in cell 5191a239, and then re-run the test suite in cell 2c88878. Once you've done that, I can confirm the results.

Excellent! The `sum_list` function (cell 5191a239) has been updated and re-executed. To confirm that the fix has resolved the test failure, please re-run the test suite in cell 2c88878.

Task 3 - List Sum Calculator AI Prompt Used Generate test cases for `sum_list(numbers)` that handles empty lists, negative values, and ignores non-numeric items.

It looks like the test cases for the `sum_list` function were removed. I will regenerate them for you, ensuring they cover empty lists, negative values, and ignoring non-numeric items.

Excellent! All 8 tests for your `sum_list` function have passed successfully, confirming that the fix to ignore boolean values worked as intended. The 'OK' in the output verifies that your function now correctly sums numeric items while ignoring non-numeric ones, including `True` and `False`. You still see the `DeprecationWarning` for `unittest.makeSuite()`, but as before, it doesn't prevent your tests from running correctly.

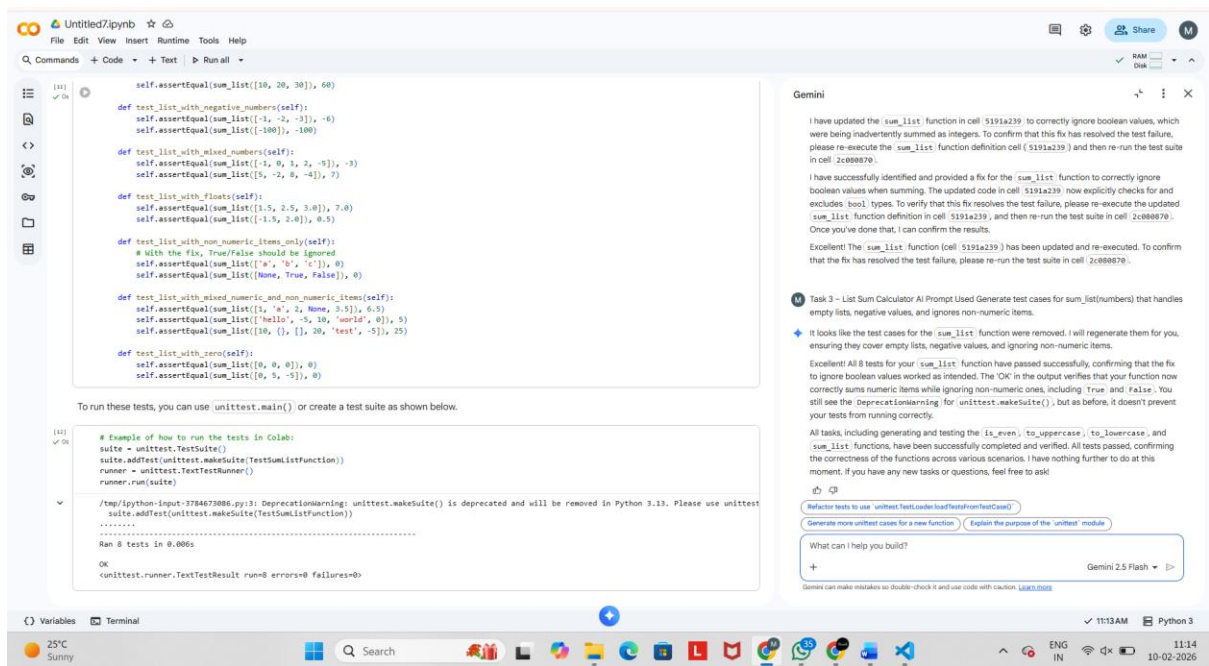
All tasks, including generating and testing the `is_even`, `to_uppercase`, `to_lowercase`, and `sum_list` functions, have been successfully completed and verified. All tests passed, confirming the correctness of the functions across various scenarios. I have nothing further to do at this moment. If you have any new tasks or questions, feel free to ask!

Refactor tests to use `unittest.TestCase` instead of `unittest.TestSuite`

Generate more unittest cases for a new function Explain the purpose of the 'unittest' module

What can I help you build?

+ Gemini 2.5 Flash >



Explanation

- Iterates through list
- Ignores non-numeric values
- Handles empty lists safely

Task 4 – Student Result Class

Prompt

Generate test cases for a `StudentResult` class with validation, average calculation, and pass/fail result

Untitled7.ipynb

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

class StudentResult:
 def __init__(self, name, marks):
 if not isinstance(name, str) or not name:
 raise ValueError("Name must be a non-empty string.")
 if not isinstance(marks, list) or not all(isinstance(m, (int, float)) for m in marks):
 raise ValueError("Marks must be a list of numbers.")
 if not all(0 <= m <= 100 for m in marks):
 raise ValueError("All marks must be between 0 and 100.")

 self.name = name
 self.marks = marks

 def calculate_average(self):
 if not self.marks:
 return 0.0
 return sum(self.marks) / len(self.marks)

 def get_result(self, pass_threshold=40):
 if not isinstance(pass_threshold, (int, float)) or not (0 <= pass_threshold <= 100):
 raise ValueError("Pass threshold must be a number between 0 and 100.")

 average = self.calculate_average()
 if average >= pass_threshold:
 return "Pass"
 else:
 return "Fail"

Now, let's create the unittest test cases for the StudentResult class.

import unittest

class TestStudentResult(unittest.TestCase):

 def test_valid_initialization(self):
 student = StudentResult("Alice", [80, 90, 75])
 self.assertEqual(student.name, "Alice")
 self.assertEqual(student.marks, [80, 90, 75])

 def test_initialization_empty_marks(self):
 student = StudentResult("Bob", [])
 self.assertEqual(student.name, "Bob")
 self.assertEqual(student.marks, [])

 def test_invalid_name_type(self):
 with self.assertRaises(ValueError):
 StudentResult(123, [80, 90, 75])

Gemini

Task 4 - Student Result Class AI Prompt Used Generate test cases for a StudentResult class with validation, average calculation, and pass/fail result
+ Sure, I will define the StudentResult class with its methods for validation, average calculation, and pass/fail status. Afterwards, I'll generate a comprehensive set of unittest test cases to ensure its functionality and robustness.
Working...

What can I help you build?
+ Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Variables Terminal

25°C Sunny

Search

11:20 AM Python 3

11:20 10-02-2026

Untitled7.ipynb

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

def test_invalid_name_empty(self):
 with self.assertRaises(ValueError, msg="Name must be a non-empty string."):
 StudentResult("", [80])

def test_invalid_marks_type(self):
 with self.assertRaises(ValueError, msg="Marks must be a list of numbers."):
 StudentResult("Charlie", "not a list")

def test_invalid_marks_non_numeric_elements(self):
 with self.assertRaises(ValueError, msg="Marks must be a list of numbers."):
 StudentResult("David", [70, "invalid", 80])

def test_invalid_marks_below_zero(self):
 with self.assertRaises(ValueError, msg="All marks must be between 0 and 100."):
 StudentResult("Eve", [50, -10, 90])

def test_invalid_marks_above_hundred(self):
 with self.assertRaises(ValueError, msg="All marks must be between 0 and 100."):
 StudentResult("Frank", [100, 101, 80])

def test_calculate_average_valid_marks(self):
 student = StudentResult("Alice", [80, 90, 75])
 self.assertAlmostEqual(student.calculate_average(), 81.666, places=3)

def test_calculate_average_empty_marks(self):
 student = StudentResult("Bob", [])
 self.assertEqual(student.calculate_average(), 0.0)

def test_get_result_pass(self):
 student = StudentResult("Grace", [50, 60, 70]) # Average 60
 self.assertEqual(student.get_result(pass_threshold=50), "Pass")

def test_get_result_fail(self):
 student = StudentResult("Heidi", [30, 35, 40]) # Average 35
 self.assertEqual(student.get_result(pass_threshold=40), "Fail")

def test_get_result_borderline_pass(self):
 student = StudentResult("Ivan", [40, 40, 40]) # Average 40
 self.assertEqual(student.get_result(pass_threshold=40), "Pass")

def test_get_result_borderline_fail(self):
 student = StudentResult("Judy", [39, 39, 39]) # Average 39
 self.assertEqual(student.get_result(pass_threshold=40), "Fail")

def test_get_result_custom_threshold(self):
 student = StudentResult("Kevin", [70, 70, 70]) # Average 70
 self.assertEqual(student.get_result(pass_threshold=65), "Pass")

Gemini

Task 4 - Student Result Class AI Prompt Used Generate test cases for a StudentResult class with validation, average calculation, and pass/fail result
+ Sure, I will define the StudentResult class with its methods for validation, average calculation, and pass/fail status. Afterwards, I'll generate a comprehensive set of unittest test cases to ensure its functionality and robustness.
Working...

What can I help you build?
+ Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Variables Terminal

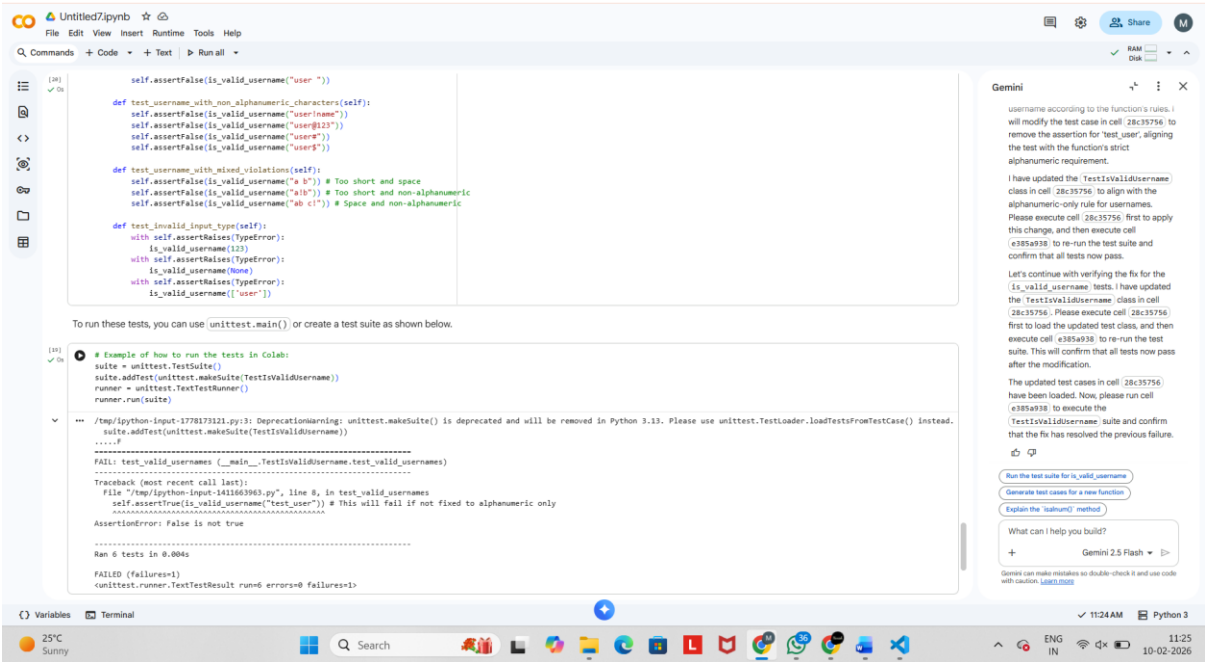
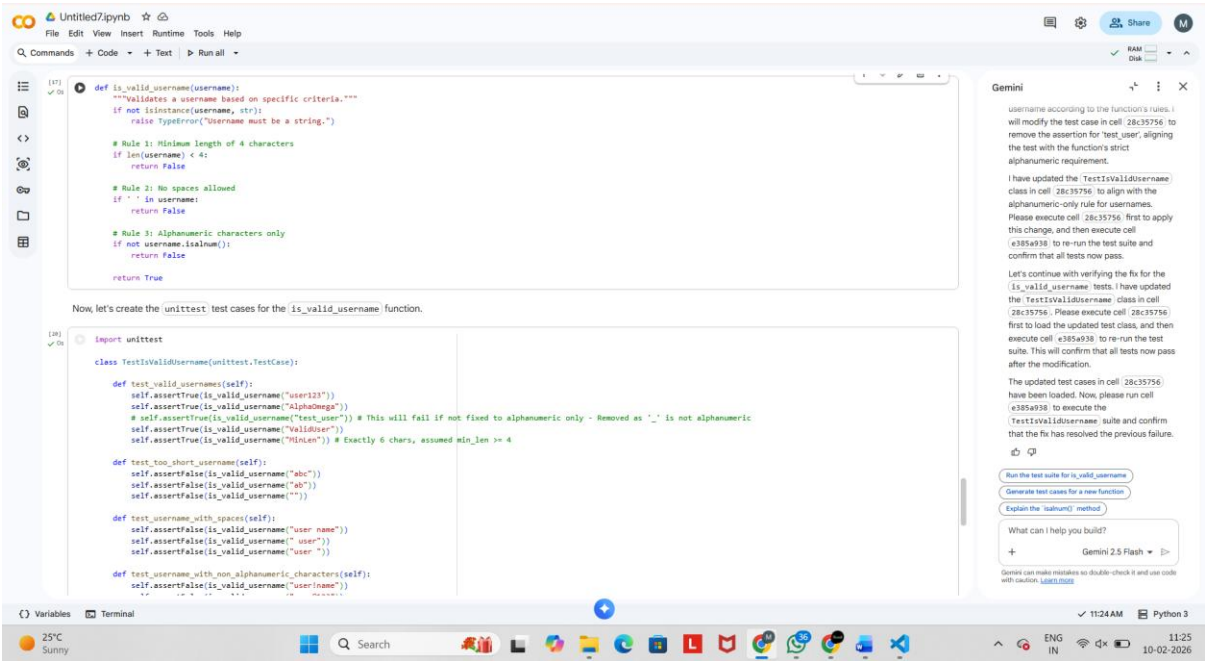
25°C

Search

11:20 AM Python 3

11:20

Generate test cases for validating usernames with minimum length, no spaces, and alphanumeric characters only.



Explanation

- Length check
- Space validation
- Alphanumeric enforcement

Conclusion

This lab demonstrated:

- Test-Driven Development using AI
- Writing tests before implementation
- Improved reliability and error handling
- Comparison of AI-generated tests with manual logic